task, the test output is formatted like the documentation of the `Cart` class:

```
> bin/rails spec SPEC_OPTS="--format=doc"

Cart
  #add_product
    adding unique products
      has two line items
      has a total price of the two items' price
    adding duplicate products
      has one line item
      has a line item with a quantity of 2
      has a total price of twice the product's price

Finished in 0.14865 seconds (files took 1.76 seconds to load)
5 examples, 0 failures
```

Also note that `rspec-rails` changes the Rails generators to create empty spec files in `spec/` instead of test files in `test/`. This means that you use all the generators and scaffolding you're used to in your normal workflow without having to worry about the wrong type of test file being created.

If all of this seems strange to you, you're not alone. It *is* strange, and the reasons RSpec is designed this way, as well as why you might want to use it, are nuanced and beyond the scope of this book. The main point all this proves is that you can replace a major part of Rails with an alternative and still get all the benefits of the rest of Rails. It's also worth noting that RSpec is popular, and you're very likely to see it in the wild.

Let's learn more about Rails' configurability by replacing another major piece of Rails—ERB templates.